
A01

OOP & JavaFX Shapes

Algorithmic Thinking
ALD (MTD.ba VZ)
SS 2026

Submission via Moodle

24 Points

This assignment covers UE1 and UE2: basic OOP concepts (classes, constructors, methods), JUnit testing, Maven project setup, encapsulation, and JavaFX shape rendering.

Submission Requirements:

- Read the `Organizational Guidelines` on Moodle, and adhere to them!
 - Submit a PDF document containing your solution idea, code, and tests for each exercise
 - Include all source files in a ZIP archive (Maven project)
 - Use the naming convention: `ALD_UE01_LASTNAME.pdf` and `ALD_UE01_LASTNAME.zip`
-

Exercise 1 (8 points) Shape Classes with Encapsulation

Implement all 7 shape classes applying the encapsulation principles from the lecture. Some classes were started in class — you can continue from there.

Each class must provide at least the following methods. All classes must implement `toString()` and `equals(Object)`.

- `Point`
 - `distanceTo(Point)`
- `Circle`
 - `getArea()`, `getPerimeter()`, `scale(double)`, `isBiggerThan(Circle)`
- `Rectangle`
 - `getArea()`, `getPerimeter()`, `getDiagonal()`
- `Line`
 - `getLength()`, `getMidpoint()`
- `Triangle`
 - `getArea()`, `getPerimeter()`
- `Ellipse`
 - `getArea()`, `getPerimeter()`
- `Star`
 - `getArea()`, `getPerimeter()`

Think carefully about what data each shape needs, how to represent it, and how to handle invalid inputs. Include a brief written description of your design decisions in your submitted PDF.

Exercise 2 (4 points) JUnit Tests for Shapes

Write JUnit 5 tests for all shape classes (again you can continue from where you left off in class).

Each test class must cover:

- `getArea()` and `getPerimeter()` (or `getLength()` for `Line`) with known expected values
- Constructor validation: negative values must be clamped to 0 (e.g., `new Circle(-5, 0, 0)` should have radius 0)
- `equals()` with both equal and different objects
- At least 2 test methods per class

Tip: Use `assertEquals(expected, actual, delta)` for floating-point comparisons with a small delta (e.g., 0.001).

Exercise 3 (8 points) JavaFX Drawing & Composition

Implement the `draw(GraphicsContext gc)` method in every shape class and create a `Drawing` class that uses composition to manage a collection of shapes.

Part A — `draw()` methods (4 points):

- Add a `draw(GraphicsContext gc)` method to each shape class
- The `draw()` methods for `Rectangle` and `Triangle` were done in class — you must implement `draw()` for `Circle`, `Line`, `Ellipse`, and `Star`
- Use the appropriate `GraphicsContext` methods: `fillOval` for `Circle`/`Ellipse`, `strokeLine` for `Line`, `fillPolygon` for `Star`
- Set the fill/stroke color from the shape's `color` field before drawing

Part B — `Drawing` class (2 points):

- Create a `Drawing` class with an `Object[]` array to store shapes (we don't have interfaces/inheritance yet!)
- Constructor takes the maximum number of shapes: `Drawing(int maxShapes)`
- `add(Object shape)` — adds a shape to the array
- `getCount()` — returns the current number of shapes
- `drawAll(GraphicsContext gc)` — iterates over all shapes and uses `instanceof` with pattern matching to call `draw()` on each shape type

Part C — `ShapeApp` (2 points):

- Create a `ShapeApp` class extending `Application`
- In `start()`: create a `Canvas(800, 600)`, get its `GraphicsContext`
- Create a `Drawing`, add at least one instance of each of the 7 shape types
- Call `drawAll()` to render everything on the canvas

Exercise 4 (4 points) Creative Scene

Design and render a creative scene using your Drawing class. This exercise is about experimentation and creativity — there is no single correct answer.

Requirements:

- Use your Drawing class to compose a scene (e.g., a house, landscape, space scene, abstract art, smiley face, city skyline, etc.)
 - Your scene must include at least one instance of each of the 7 shape types: Point, Circle, Rectangle, Line, Triangle, Ellipse, Star
 - Experiment with colors: `Color.RED`, `Color.rgb(128, 0, 255)`, `Color.CORNFLOWERBLUE`, etc.
 - Include a screenshot of the running application in your submitted PDF
 - Briefly describe (2–3 sentences) what your scene depicts and which shapes you used for what
-